

Autonomous QA & Bug Reproducer

Plan of Action: Building on the Solari SDK Cookbook

ARCHITECTURE BLUEPRINT	SDK: SOLARI V1.X	ENVIRONMENT: NODE.JS / TYPESCRIPT	SCOPE: 4-PHASE DELIVERY
---------------------------	---------------------	--------------------------------------	----------------------------

Executive Overview

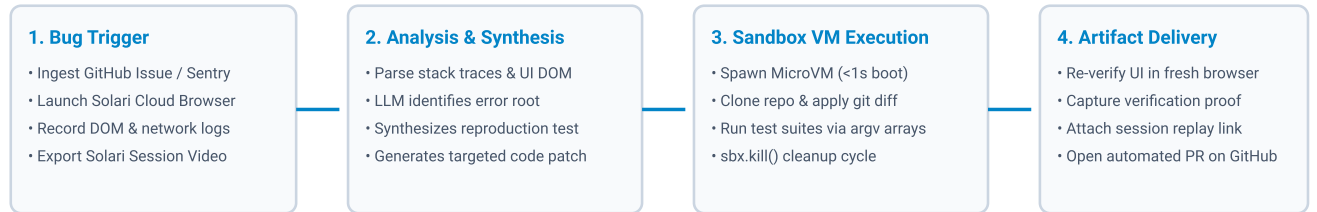
This document outlines the end-to-end plan to engineer an **Autonomous Bug Reproduction and Regression Verification Agent** leveraging primitives from `solari-sdk/solari-cookbook`. The solution combines headless browser automation with recording, isolated microVM compute for patch testing, and desktop verification into a unified QA pipeline.

1. Core Solari SDK Primitives Allocation

The system decomposes agentic quality assurance into three distinct infrastructural pillars provided by Solari:

Cloud Browser Role: Issue reproduction & recording Automated Playwright sessions with session recording, network inspection, and stealth capabilities to trace UI faults.	Sandbox (MicroVM) Role: Code repair & unit verification Sub-second ephemeral Linux environments used to clone repositories, apply LLM code patches, and execute test suites.	Desktop (X11) Role: Complex GUI validation VNC-accessible desktop environment for verifying deep OS-level desktop applications and multi-window workflows.
---	--	--

2. System Architecture & Information Flow



Core Architecture Objective

Eliminate "works on my machine" triage bottlenecks by automating reproduction across cloud-isolated runtimes. Engineers receive verified code patches accompanied by deterministic recording replays.

3. Phased Implementation Roadmap

The roll-out schedule spans four structured phases, prioritizing security, deterministic isolation, and developer experience.

Phase	Objectives	Key Solari Operations	Deliverables
Phase 1: Setup & Primitives	Establish API orchestrator; build client wrapper for Cloud Browser and Sandbox lifecycles.	<code>SolariBrowser.launch({ recording: true })</code> <code>SandboxClient.create({ template: "base" })</code>	Base harness and connection test suite.
Phase 2: Reproducer Engine	Automate headless browser tracing; extract console logs, network waterfalls, and DOM errors.	Playwright CDP binding, page error telemetry, and video replay generation.	Reproduction harness with video export.
Phase 3: VM Sandbox Runner	Build sandboxed microVM test runner; apply git diffs and execute regression tests safely.	<code>sbx.commands.run()</code> with explicit argument vectors; strict <code>sbx.kill()</code> cleanup.	Isolated code executor and patch verifier.
Phase 4: Agentic Loop & CI/CD	Integrate LLM feedback loop for automated diff generation, PR comments, and GitHub Actions trigger.	Dual-mode verification combining Sandbox unit tests with fresh Cloud Browser replay checks.	End-to-end autonomous QA bot integration.

4. Step-by-Step Technical Execution Workflow

Step 1: Ingestion & Browser Recording - Ingest target URL and steps to reproduce. - Launch Solari Browser with <code>stealth: true</code> and session recording enabled. - Replay input events, intercept network responses, and capture uncaught exceptions. - Obtain signed video replay URL upon session termination.	Step 2: MicroVM Sandbox Execution - Initialize ephemeral Linux microVM in under 1 second. - Run <code>git clone</code> over SSH or GitHub App token. - LLM synthesizes failing unit test and applies candidate diff via <code>commands.run("git", { args: ["apply", "patch.diff"] })</code>. - Assert <code>exit_code === 0</code> across test runs.
Step 3: Verification & Re-testing - Deploy candidate build to a temporary preview container. - Launch fresh Cloud Browser session against preview environment. - Verify regression resolution without new client-side faults. - Confirm DOM stability and visual alignment.	Step 4: PR & Artifact Publication - Commit tested patch to a designated feature branch. - Generate markdown summary with test output logs. - Embed Solari video replay link directly in pull request. - Release all cloud resources with guaranteed disposal.

Security & Isolation Guarantee

All code execution occurs inside dedicated MicroVMs with hardware-assisted virtualization. Untrusted dependencies or hostile test scripts have zero access to the host network or internal infrastructure.

5. Critical Operational Rules (From Solari Cookbook)

Adherence to SDK lifecycle patterns is vital to prevent orphaned VMs, budget leakage, and session hangs:

- 1. MicroVM Lifecycle: Use `sbx.kill()` , Never Just `close()`**

Calling `close()` merely terminates the client-side communication channel while leaving the microVM active until its configured idle timeout. The runner must implement an explicit `try...finally` block invoking `await sbx.kill()` to immediately tear down the instance and prevent billed idle time.
- 2. Explicit Argument Vectors (No Shell Interpretation)**

Solari commands do not execute inside a shell wrapper by default. Passing concatenated strings like `"npm test -- filter=unit"` will trigger invalid command errors. Commands must pass explicit arrays: `{ args: ["test", "-- filter=unit"] }`, or explicitly invoke `sh -c` when shell operators (pipes, redirects) are required.
- 3. Replay Availability & Asynchronous Finalization**

Session recordings are finalized and uploaded asynchronously upon browser session closure. A polling backoff of ~25–30 seconds must be respected before asserting replay URLs in GitHub notifications.

6. Success Metrics & Verification KPI Grid

Key Performance Indicator	Baseline Manual Process	Target Solari Agent System	Evaluation Standard
Reproduction Turnaround	45–90 minutes per bug	< 3.5 minutes end-to-end	Measured from webhook receipt to video replay generation.
Test Environment Provisioning	5–15 minutes (CI pipeline)	< 1.5 seconds per VM	Solari base template instantiation latency.
Patch Verification Accuracy	Variable / manual review	100% deterministic (exit code 0)	Candidate patch must pass suite in isolated clean-slate VM.
Evidence Completeness	Static screenshots & text logs	Interactive video + DOM traces	Signed Solari replay attached to every generated PR.

- Next Actions & Immediate Kickoff**

 1. Initialize repo with `@solarisdk/browser` and `@solarisdk/sandbox` .
 2. Set up GitHub webhook receiver for issue triggers.
 3. Validate `sbx.kill()` cleanup routines with a stress-test suite of 25 concurrent sandbox jobs.

